Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 16

All-Pairs Shortest Paths (Part 1)

Πανζευκτικές Ελαφρύτατες Διαδρομές (Μέρος 1)

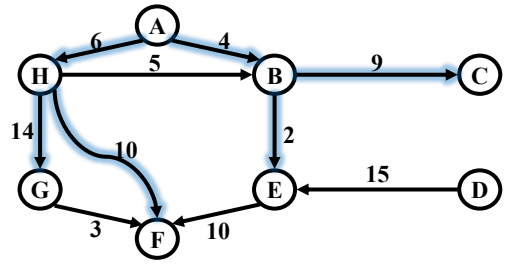
Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, E. D. Demaine, and M. Christodoulakis

1

Single-Source Shortest Paths (Ομοαφετηριακές Ελαφρύτατες Διαδρομές)

- General case
  - Bellman-Ford algorithm: $O(VE)$
- Nonnegative edge weights
  - Dijkstra's algorithm with Fibonacci heap: $O(V \lg V + E)$
- Directed acyclic graph (DAG)
  - One pass of Bellman-Ford in topological order: $O(V + E)$



2

All-Pairs Shortest Paths
(Πανζευκτικές Ελαφρύτατες Διαδρομές)

- Input: Directed graph $G = (V, E)$, where $V = \{1, 2, \ldots, n\}$, with edge-weight function $w: \mathrm{E} \to \mathbb{R}$
- Output: $n \times n$ matrix $D = (d_{ij})$ of shortest-path lengths $\delta(i, j)$ for all $i, j \in V$
- Most algorithms we'll see use an **adjacency-matrix** (**πίνακας γειτνίασης**) $W = (w_{ij})$

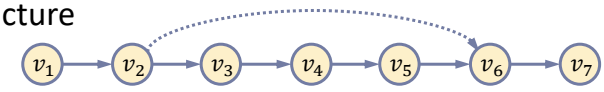
$$w_{ij} = \begin{cases} 0 & \text{if } i = j \\ \text{weight of edge } (i,j) & \text{if } i \neq j \text{ and } (i,j) \in E \\ \infty & \text{if } i \neq j \text{ and } (i,j) \notin E \end{cases}$$

All-Pairs Shortest Paths – Ideas

- General Graphs
  - Run Bellman-Ford once from each vertex
  - Running time: $O(V^2E)$
  - Dense graph: $O(V^4)$ (*Why?* $E = \Theta(V^2)$)
- Nonnegative edge weights
  - Run Dijkstra's algorithm once from each vertex
  - Running time with Fibonacci heap: $O(V^2 \lg V + VE)$
    - $O(V^3)$ if dense

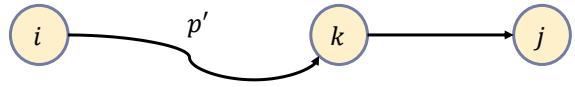
Dynamic Programming

- Optimal Substructure
  - *Which is?*



  Subpaths of shortest paths are also shortest paths

- Consider the shortest path p from vertex i to vertex j
  - Suppose p contains **at most** m edges and no negative cycles
  - If $i = j$ then $w(p) = 0$ and p has no edges
  - If $i \neq j$ then there is a path p' from $i \rightsquigarrow k$ and edge $k \rightarrow j$ such that p' has at most $m - 1$ edges
  - p' is a shortest path and $\delta(i,j) = \delta(i,k) + w_{kj}$ (*Why? Opt. Sub.*)



Building a Recursive Solution

- Consider the $n \times n$ adjacency matrix $W = (w_{ij})$ of the directed graph, and define

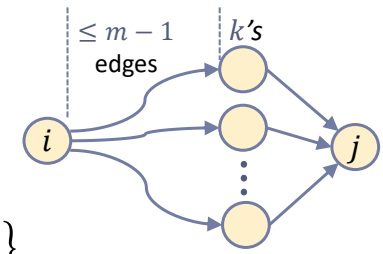
  $l_{ij}^{(m)}$ = weight of a shortest path from i to j that uses $\leq m$ edges

- For $m = 0$

$$l_{ij}^{(0)} = \begin{cases} 0 & \text{if } i = j \\ \infty & \text{if } i \neq j \end{cases}$$

  and for $m \geq 1$

$$l_{ij}^{(m)} = \min_{1 \leq k \leq n} \left\{ l_{ik}^{(m-1)} + w_{kj} \right\}$$



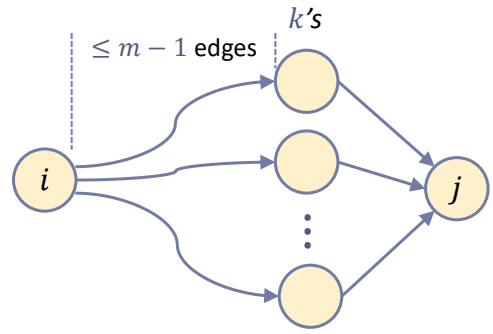
k is all possible **predecessors** (**προκάτοχοι**) of j

- *What happens when $m = 1$?*

$$l_{ij}^{(1)} = w_{ij}$$

Building a Recursive Solution

- $l_{ij}^{(m)} = \min_{1 \le k \le n} \left\{ l_{ik}^{(m-1)} + w_{kj} \right\}$



- **Relaxation!**

for $k \leftarrow 1$ to n
  do if $l_{ij} > l_{ik} + w_{kj}$
    then $l_{ij} \leftarrow l_{ik} + w_{kj}$

- Note: No negative-weight cycles implies

$\delta(i,j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} = \cdots$ *Why? Shortest path from i to j has at most $n-1$ edges*

Computing a Solution Bottom Up

- Compute $L^{(1)}, L^{(2)}, \ldots, L^{(n-1)}$
  - Start with $L^{(1)} = W$ since $l_{ij}^{(1)} = w_{ij}$
  - Go from $L^{(m-1)}$ to $L^{(m)}$

EXTEND(L, W, n) {
Create L', an $n \times n$ matrix
for $i \leftarrow 1$ to n
  for $j \leftarrow 1$ to n
    $l'_{ij} \leftarrow \infty$
    for $k \leftarrow 1$ to n
      $l'_{ij} \leftarrow \min(l'_{ij}, l_{ik} + w_{kj})$
return L' }

SLOW-APSP(W, n) {
$L^{(1)} \leftarrow W$
for $m \leftarrow 2$ to $n-1$
  $L^{(m)} \leftarrow \text{EXTEND}(L^{(m-1)}, W, n)$
return $L^{(n-1)}$
}

What are the running times?
EXTEND: $\Theta(n^3)$
SLOW-APSP: $\Theta(n^4)$

EXTEND is like Matrix Multiplication

- What if we map "min" → "sum" and "+" → "·"?

$$l_{ij} = \min_{1 \le k \le n} \{l_{ik} + w_{kj}\} \quad \rightarrow \quad c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

EXTEND(L, W, n) {	**MATRIX_MULTIPLY**(A, B, n) {
Create L', an $n \times n$ matrix	Create C, an $n \times n$ matrix
for $i \leftarrow 1$ to n	for $i \leftarrow 1$ to n
for $j \leftarrow 1$ to n	for $j \leftarrow 1$ to n
$l'_{ij} \leftarrow \infty$	$c_{ij} \leftarrow 0$
for $k \leftarrow 1$ to n	for $k \leftarrow 1$ to n
$l'_{ij} \leftarrow \min(l'_{ij}, l_{ik} + w_{kj})$	$c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$
return L' }	return C }

EXTEND is like Matrix Multiplication

- What if we map "min" → "sum" and "+" → "·"?

$$l_{ij} = \min_{1 \le k \le n} \{l_{ik} + w_{kj}\} \quad \rightarrow \quad c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

- Consequently, we can compute

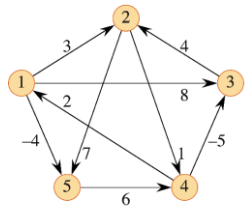
$$L^{(1)} = L^{(0)} \cdot W = W^1$$
$$L^{(2)} = L^{(1)} \cdot W = W^2$$
$$\ldots$$
$$L^{(n-1)} = L^{(n-2)} \cdot W = W^{n-1}$$

yielding

$$L^{(n-1)} = \left(\delta(i,j)\right)$$

Why? Largest possible shortest path has length $n-1$

Example of All-Pairs Shortest Path



$$W = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

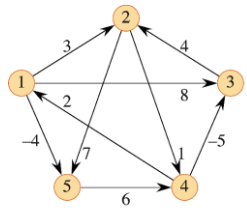
$$L^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

- Intuition: To find (i, j) entry in $L^{(2)}$, process row i in $L^{(1)}$ and column j in W similar to matrix multiplication but using sum and min

- Example:

$$\begin{aligned} l_{34}^{(1)} &= \min\left\{l_{31}^{(0)} + w_{14}, l_{32}^{(0)} + w_{24}, l_{33}^{(0)} + w_{34}, l_{34}^{(0)} + w_{44}, l_{35}^{(0)} + w_{54}\right\} \\ &= \min\{\infty + \infty, 4 + 1, 0 + \infty, \infty + 0, \infty + 6\} \\ &= 5 \end{aligned}$$

Example of All-Pairs Shortest Path



$$W = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$L^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad L^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 2 & -4 \\ 3 & 0 & -4 & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & \infty & 1 & 6 & 0 \end{pmatrix}$$

$$L^{(3)} = \begin{pmatrix} 0 & 3 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad L^{(4)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

Improved Matrix Multiplication Algorithm

- Our goal is to compute $L^{(n-1)}$ as fast as we can – not all intermediate $L^{(1)}, L^{(2)}, \ldots, L^{(n-2)}$
- In matrix multiplication, if we want to compute A^{n-1}, we can compute $A^2, A^4, A^8, \ldots$
- Consequently, we can compute $L^{(n-1)}$ with $\lceil \lg(n-1) \rceil$ **repeated squarings** (**επαναληπτικοί τετραγωνισμοί**)

$$L^{(1)} = W$$
$$L^{(2)} = W^2 = W \cdot W$$
$$L^{(4)} = W^4 = W^2 \cdot W^2$$
$$L^{(8)} = W^8 = W^4 \cdot W^4$$
$$\ldots$$

Improved Matrix Multiplication Algorithm

- Q: If $n-1$ is not a power 2, does the repeating squaring give the correct result?
- A: $L^{n-1} = L^n = L^{n+1} = \cdots$ *Why?* $\delta(i,j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = \cdots$

FASTER-APSP(W, n) {
$L^{(1)} \leftarrow W$
$m \leftarrow 1$
while $m < n-1$
  $L^{(2m)} \leftarrow \text{EXTEND}\left(L^{(m)}, L^{(m)}, n\right)$
  $m \leftarrow 2m$
return $L^{(m)}$
}

What is the running time?
$\Theta(n^3 \lg n)$

How can we detect negative-weight cycles?
Check the diagonal for negative values